

CNL Final Report: Trumproxy

Group 6

鄭博允 毛翊蓁 章譯中
B11902038 B10902103 B11902039

簡昶諺 徐子翔 陳垣豪
B11902020 B11902034 B11902172

Abstract—Trumproxy is an experimental system designed to simulate network-level customs by enforcing tariffs on data packets based on their geographic origin. Operating within a Docker container, Trumproxy allows administrators to manage network traffic through a user-friendly frontend interface. Users connect via a WireGuard VPN tunnel, enabling the interception and manipulation of network traffic using mitmproxy. The system’s architecture comprises four primary components: the Main Module, Proxy, Backend API Server, and Frontend Interface. This modular design ensures broad platform compatibility and flexible deployment. Trumproxy’s workflow involves intercepting network requests, determining their country of origin via GeoIP lookups, and applying predefined tariff rules, such as delaying or dropping packets. This framework provides a controlled environment to study the implications of network tariffs and their enforcement.

I. INTRODUCTION

In an era of increasing globalization, the concept of applying tariffs has extended beyond physical goods to the digital realm. Network tariffs, akin to customs duties, involve regulating data flow based on its origin, with potential applications in cybersecurity, data sovereignty, and digital trade policies. Understanding the practical implications of such tariffs necessitates a robust platform for simulation and analysis.

Trumproxy is developed to address this need by providing a system that emulates network-level customs enforcement. By leveraging technologies such as Docker, WireGuard, and mitmproxy, Trumproxy enables administrators to impose and monitor tariffs on data packets based on their geographic source.

II. SYSTEM DESIGN

This chapter details the implementation of Trumproxy, an experimental system designed to explore the concept of network tariffs. An overview diagram is provided in Fig. 1 for enhanced clarity.

Trumproxy operates within a Docker container on the administrator’s device. The administrator can manage Trumproxy through a frontend interface accessible via localhost:5000. Users on the same Local Area Network (LAN) as the administrator (e.g., connected to the same Wi-Fi network) can connect to Trumproxy by establishing a VPN tunnel. This connection requires users to first install the WireGuard VPN client, which is widely available across various platforms.

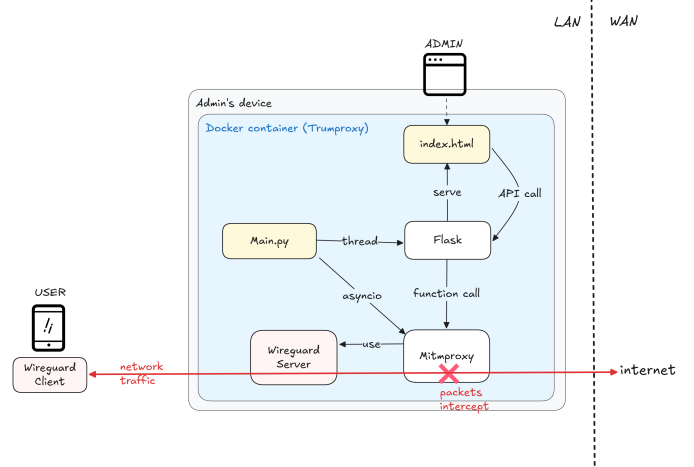


Fig. 1. Architecture of Trumproxy

A. Tech Stack

The implementation of Trumproxy leverages a modular and containerized technology stack to ensure broad platform compatibility and flexible deployment. The frontend is developed using standard web technologies—HTML, CSS, and JavaScript—to provide a lightweight and accessible administrative interface. The backend API is built with Python, utilizing the Flask framework to facilitate efficient RESTful communication with the proxy layer.

The core proxy functionality is achieved through the integration of mitmproxy, a robust intercepting proxy, with WireGuard, a high-performance VPN protocol renowned for its simplicity and cross-platform support. This combination enables Trumproxy to transparently intercept, inspect, and manipulate network traffic. To streamline deployment across diverse environments, the entire application is containerized using Docker, which enhances reproducibility and scalability.

B. Architecture

The system architecture of Trumproxy comprises four primary components:

1) *Main Module*: The main module is a Python script responsible for concurrently running both the Backend API Server and the Proxy. This concurrent execution is necessary

because the Backend directly invokes functions provided by the Proxy. To achieve this, Python’s asyncio event loop is employed to manage the asynchronous mitmproxy process, while the Flask server operates in a separate thread via the standard threading module. This concurrency model ensures non-blocking execution for both services, allowing the system to remain responsive to administrative requests while continuously monitoring and manipulating network traffic in real-time.

2) *Proxy*: The Proxy operates based on mitmproxy, running within a WireGuard-configured VPN environment to provide the user with a VPN tunnel. This allows us to intercept the user’s network traffic, which is then forwarded to mitmproxy for subsequent packet processing, such as delaying or dropping.

GeoIP lookups are performed using the MaxMind GeoIP2 database to determine the country of origin for each packet. The proxy and backend server are separated to allow other interfaces to utilize the proxy module via Python directly.

3) *Backend API Server*: The Backend API Server is implemented using Flask and exposes endpoints for managing tariff rules, retrieving cached packets, and interfacing with the proxy engine. It functions as a middleware layer between the frontend and the proxy.

4) *Frontend Interface*: A user interface has been meticulously designed to empower administrators with the ability to configure tariff rules based on geographic IP sources and to actively monitor intercepted network activity. This frontend is dynamically served using Flask’s `render_template()` function. Figures 2 and 3 provide illustrative snapshots of Trumproxy’s intuitive user interface.

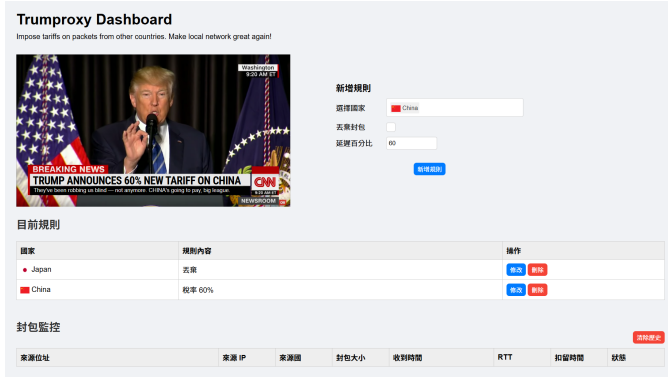


Fig. 2. Main Dashboard of Trumproxy, where tariff rules can be configured here.

C. Workflow

The workflow of Trumproxy simulates a network-level customs system inspired by geopolitical tariff practices. The stepwise operation is as follows:

- 1) A client device initiates a request to an external website via the Trumproxy VPN tunnel.
- 2) The mitmproxy component intercepts the response from the remote server.

目前規則

國家	規則內容	操作	
✖ South Korea	封包	<button>新增</button>	<button>刪除</button>

封包監控

清除歷史

來源位址	來源 IP	來源國	封包大小	收到時間	RTT	扣留時間	狀態
https://www.gov.kr/portal/foreigner/en	125.60.35.230	✖ South Korea	22660	2025/5/28 下午3:07:25	0.568ms	0s	<button>封包</button>
https://www.gov.kr/portal/foreigner/en	125.60.35.230	✖ South Korea	22660	2025/5/28 下午3:07:26	0.5974ms	0s	<button>封包</button>
https://www.gov.kr/portal/foreigner/en	125.60.35.230	✖ South Korea	22660	2025/5/28 下午3:07:26	0.4146ms	0s	<button>封包</button>
https://www.gov.kr/portal/foreigner/en	125.60.35.230	✖ South Korea	22660	2025/5/28 下午3:07:26	0.3684ms	0s	<button>封包</button>
https://www.gov.kr/portal/foreigner/en	125.60.35.230	✖ South Korea	22660	2025/5/28 下午3:07:27	0.1201ms	0s	<button>封包</button>

Fig. 3. Packet Table to display currently delayed or dropped packets.

- 3) The source IP address of the response is evaluated using GeoIP2 to determine its country of origin.

- 4) If a country-specific tariff rule exists:

- Delay Rule: The packet is temporarily held in memory using a sleep-based delay mechanism before being forwarded to the user.
- Drop Rule: The packet is discarded, and metadata is retained for monitoring and reporting.

- 5) The proxy logs all delayed or dropped packets, which are visualized on the frontend dashboard.

This framework enables a controlled and observable emulation of network tariff enforcement.

III. PROBLEMS & METHODS

In this chapter, we discuss the major technical challenges encountered during the development of Trumproxy, as well as the methodologies adopted to address them. The primary areas of concern include inter-process communication between the proxy and backend server, redirecting client traffic through the proxy, and enforcing packet delays in real (physical) time.

A. Inter-process communication between the proxy and backend server

A key architectural challenge was facilitating real-time communication between the proxy layer, which inspects and manipulates packets, and the backend API server, which provides administrative control and visualization. Since the proxy runs in an asynchronous context and the Flask API server operates synchronously, achieving concurrency without blocking operations was non-trivial.

To address this, the system was designed to execute the mitmproxy logic using Python’s asyncio framework, allowing it to run an event-driven packet inspection loop. Simultaneously, the Flask API server was launched on a separate thread using the threading module. Shared data structures (e.g., packet logs and tariff rules) were managed using thread-safe dictionaries, which were guarded by locks when necessary to ensure data integrity across concurrency boundaries. This hybrid concurrency model allowed the system to simultaneously handle network traffic and administrative commands without interference or latency.

B. Redirecting client traffic through the proxy

Another significant challenge was how to route all client device traffic through Trumproxy, which is essential for intercepting and analyzing cross-border data flow. To accomplish this, a VPN-based approach was adopted, utilizing WireGuard, a modern and efficient tunneling protocol known for its ease of configuration and broad platform support.

Client devices were required to install the WireGuard client and import a configuration file generated by the server. This VPN configuration ensured that all outgoing traffic was securely tunneled to the proxy host. Once traffic entered the proxy, mitmproxy intercepted HTTPS connections by performing man-in-the-middle inspection, enabled through the installation of a custom Certificate Authority (CA) via mitm.it [1]. While this approach was effective on most platforms, it faced limitations on iOS, where only the Safari browser consistently respected the installed VPN certificate—highlighting the need for further research into mobile VPN and certificate propagation behaviors.

C. Enforcing packet delays in real time

Implementing a realistic network tariff model necessitated the ability to delay packets in real-world time, simulating congestion or customs clearance. However, mitmproxy’s architecture is event-driven and primarily designed for real-time packet forwarding, which presented a challenge for introducing latency.

To simulate physical time delays, packets that matched a configured tariff rule were cached in memory and held using Python’s `time.sleep()`. After the specified delay duration, the packets were released and forwarded to the user. While this approach effectively introduced observable latency, it came with trade-offs—particularly high memory usage if numerous packets were cached simultaneously. As a mitigation strategy, the system periodically clears expired or completed packet entries, but comprehensive stress testing under high-throughput scenarios remains an open area for future enhancement.

IV. RESULTS AND DISCUSSION

A. System Achievements

The development and deployment of Trumproxy demonstrated a novel idea about network-level tariffs. While the system remains in a prototype stage, we can see considerable potential in Trumproxy, offering a lightweight yet practical approach to managing network flow control.

At this stage, Trumproxy provides complete functionality to drop or delay packets based on country-specific rules. These rules are enforced in real-time through a coordinated pipeline that combines VPN tunneling via WireGuard, packet interception using mitmproxy, and GeoIP-based source identification.

Trumproxy also features an intuitive user interface that allows administrators to configure tariff rules and monitor network activity without requiring in-depth technical knowledge, making the system more accessible to those with no prior network engineering domain knowledge.

B. Limitation

Despite Trumproxy’s effectiveness in emulating a network-level tariff system, several limitations constrain its current deployment and highlight areas for further refinement. One notable limitation is the requirement for users to manually install and configure the WireGuard VPN client. Although WireGuard is lightweight and offers broad platform support, the absence of automated configuration introduces a usability barrier, particularly for non-technical users. Providing pre-configured profiles or integrating an automated setup script could substantially improve accessibility and facilitate broader adoption.

A further challenge stems from Trumproxy’s reliance on GeoIP-based lookup to determine the geographic origin of network traffic. In real-world scenarios, many websites and services are delivered via content delivery networks (CDNs), which frequently obscure the actual location of the originating server. As a result, geolocation may be inaccurate, leading to incorrect enforcement or bypassing of tariff rules. While GeoIP databases such as MaxMind’s offer relatively high accuracy under typical conditions, their effectiveness diminishes in CDN-heavy environments where IP attribution is inherently ambiguous.

From a performance standpoint, Trumproxy temporarily holds delayed packets in memory before forwarding or discarding them. This memory-based retention model simplifies implementation but raises concerns regarding scalability. In high-throughput scenarios, the accumulation of retained packets may result in excessive memory consumption, potentially degrading system responsiveness or causing instability. Although a periodic cleanup mechanism has been implemented, its efficacy under sustained traffic loads has not yet been systematically evaluated.

Device compatibility also presents a limitation, particularly on iOS platforms. During testing, it was observed that only the Safari browser correctly routed traffic through the VPN tunnel, while other applications failed to do so. This issue is likely attributable to iOS’s handling of VPN certificates and network extension behavior, which may restrict non-browser traffic from utilizing the VPN. As a result, Trumproxy’s monitoring capabilities on iOS are significantly limited and may require platform-specific solutions to ensure broader applicability.

Finally, the current version of Trumproxy lacks persistent storage for tariff rules and packet metadata. All configuration states and logs are stored in volatile memory and are lost upon container shutdown or restart. This design hinders support for long-term usage scenarios such as historical analysis, auditing, and session recovery. To overcome this, future iterations of the system should incorporate persistent storage mechanisms, such as lightweight databases or file-based logging, to enable durability and continuity.

V. FUTURE WORK

Looking ahead, Trumproxy has the potential to evolve from a local prototype into a practical platform for large-scale network governance. One promising direction is to expand its

capability to handle all cross-border traffic, aligning with how national firewalls regulate international internet exchanges. By incorporating such functionality, Trumproxy could be deployed as part of a comprehensive infrastructure for enforcing digital sovereignty and protecting information borders.

In addition, the system can implement more realistic forms of network tariffs. Rather than merely delaying or dropping packets, future versions could support bandwidth throttling to simulate congestion or introduce payment-based access to specific content, services, or geographic regions. This would allow administrators or governments to apply differentiated access policies based on economic, political, or security considerations, making the system viable for real-world deployment.

Trumproxy may also serve as a strategic tool for enforcing digital policies in response to foreign threats or as part of broader national cybersecurity measures. Through flexible rule configuration, it can help control how data flows between nations, restrict the influence of foreign platforms, or impose penalties on traffic from designated regions.

To achieve these goals, future development will focus on enhancing scalability, introducing persistent rules and packet management, and streamlining client onboarding. These enhancements will make Trumproxy more stable, configurable, and accessible, supporting its use not only in local environments but also in national or global-scale deployments.

VI. CONTRIBUTIONS

TABLE I
CONTRIBUTIONS

Student ID	Name	Contribution (%)
B11902038	鄭博允	18%
B10902103	毛翊蓁	18%
B11902039	章譯中	18%
B11902020	簡廷諤	18%
B11902034	徐子翔	10%
B11902172	陳垣豪	18%

REFERENCES

- [1] mitmproxy. <https://mitmproxy.org/>. Accessed: 2025-6-3.
- [2] WireGuard. <https://www.wireguard.com/>. Accessed: 2025-6-3.
- [3] Mitmproxy 架設 WireGuard VPN 監測手機封包. <https://medium.com/@jincoco/mitmproxy-%E6%9E%B6%E8%A8%AD-wireguard-vpn-%E7%9B%A3%E6%B8%AC%E6%89%8B%E6%A9%9F%E5%B0%81%E5%8C%85-d04efa90d941>. Accessed: 2025-6-3.